



SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example. Answer: Testing should be treated as a systematic engineering activity rather than random or ad-hoc work. For a banking application, the team should first analyze requirements and design test cases using techniques such as equivalence partitioning, boundary value analysis, and decision tables. Testing tools can be used for automation, defect tracking, and regression testing. For example, a banking login should be tested with valid, invalid, empty, and boundary inputs. A planned testing process improves defect detection, reduces repeated errors, and increases software quality.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes. Answer : A structured testing process should be followed throughout the	3



	SDLC. First, testers analyze requirements, then prepare a test plan, design test cases, execute them, report defects, and perform regression testing after fixes. For ATM withdrawal, test cases should include sufficient balance, insufficient balance, invalid PIN, invalid amount, and withdrawal-limit conditions. Following the same process for every release reduces missed defects and improves consistency. Thus, a defined testing process makes the ATM system more reliable and reduces failures.	
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples. Answer :- An error is a human mistake made during software development, a defect is the fault introduced into the software because of that error, and a failure occurs when the software produces an incorrect result during execution. For example, if a developer writes an incorrect transaction calculation, it becomes a defect, and when the application displays the wrong transaction amount, it results in a failure. Understanding these differences helps testers report problems clearly and helps developers identify and fix the actual cause.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example. Answer:- Defect clustering means that a large number of defects are	3



	usually concentrated in a small number of modules. Testers should analyze previous defect reports and identify high-risk modules. These modules should receive higher testing priority, including regression and detailed functional testing. For example, if most defects in a banking application occur in the payment module, testers should give more attention to that module. Using defect clustering helps utilize testing resources effectively and improves defect detection.	
5	classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements. Answer:- Defects should be classified according to their type, such as functional, performance, usability, security, interface, and logical defects, and stored in a defect repository. Testers can study previous defects to identify repeated problems and high-risk areas. For example, if payment defects repeatedly occur because of incorrect validation, additional validation test cases can be created. The repository also supports regression testing and helps prevent the same defects from appearing in future releases. This improves overall software quality.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures. Answer: To identify the cause of frequent crashes during fund transfers, testers should first reproduce the failure under different conditions. They should test network interruptions, invalid account details, insufficient balance, large transaction amounts, and database failures. Logs and error messages can help identify the root cause. After fixing the defect, regression testing should be performed to ensure that existing functions are not affected. Systematic testing and proper error handling can reduce	3



	future crashes and improve application reliability.	
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively. Answer:- Testing and debugging are different activities. Testing is used to identify failures and defects, whereas debugging is used by developers to locate the cause of a defect and correct it. For example, testing may show that the account balance is incorrect, while debugging identifies the wrong calculation statement responsible for it. Confusing these activities can delay defect resolution. Therefore, testers should report failures clearly and developers should use debugging techniques to find and fix their causes.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage. Answer:- Cross-browser testing ensures that a web application works correctly on different browsers and versions. Testers should test important functions, layouts, forms, and transactions on browsers such as Chrome, Firefox, Edge, and Safari. Automated tools such as Selenium can improve testing coverage. Different screen sizes and browser versions should also be considered. Systematic cross-browser testing helps identify compatibility problems and ensures that users receive consistent functionality across different environments.	3



9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy. Answer:- Incorrect salary calculations are mainly functional or logical defects because the program does not produce the expected result. Testers should examine salary formulas, deductions, overtime, allowances, and boundary values. Equivalence partitioning and boundary value analysis can be used to test different salary conditions. Unit and regression testing should be performed after fixing the defect. Proper classification helps testers select suitable testing techniques and prevents similar calculation errors in future releases.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness. Answer:- Random testing may miss important conditions because it does not provide systematic coverage. A better approach is to design test cases based on requirements using techniques such as equivalence partitioning, boundary value analysis, decision tables, and state transition testing. Each test case should specify inputs, expected results, actual results, and pass/fail status. For example, a banking transaction should include valid, invalid, minimum, maximum, and exceptional inputs. Structured test design improves coverage and defect detection.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting. Answer:- Data inconsistency in a healthcare application may occur because	3

	of incorrect requirements, database errors, interface problems, or improper data validation. Testers should identify where the data becomes inconsistent between different modules and verify the complete data flow. Integration testing should be used to check communication between modules, while data integrity testing should verify that information is stored and retrieved correctly. For example, when a patient's information is updated, the same information should appear correctly in all related modules. Defects should be properly recorded with their details and evidence. Systematic testing and effective defect tracking .	
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect	3



patterns can be identified. Suggest improvements to prevent recurring defects.

Answer :-

A defect repository stores information about defects identified during different stages and releases of software. Testers can analyze the repository based on defect type, severity, affected module, cause, and resolution to identify common patterns. If the same defect appears repeatedly, additional test cases should be created to verify that functionality during future releases. For example, repeated transaction defects can be converted into regression test cases. Repository data can also help identify high-risk modules and guide future test planning. Therefore, effective use of a defect repository helps identify defect patterns, prevent recurring problems, and improve software quality.

13

A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

3

Answer:-

A web-based learning system may fail during peak usage because high-load scenarios were not properly included in testing. The test plan should include performance testing, load testing, and stress testing to identify the system's limitations. Testers should simulate many students accessing the system simultaneously and monitor response time, throughput, resource usage, and error rates. Important functions such as login, content access, and assignment submission should be tested under heavy load. The results should be analyzed to identify performance bottlenecks. Better test planning and coverage can improve system reliability and prevent failures during peak usage.

14

A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

3

Answer:-



	Testers should be involved from the early stages of the Software Development Life Cycle rather than only after development is completed. During requirements analysis, testers can identify unclear, incomplete, or conflicting requirements. During the design stage, they can identify risks and prepare test scenarios in advance. For example, a tester may identify missing invalid-login conditions before coding begins. Early detection of defects reduces rework, development cost, and project delays. Regular communication between testers and developers improves collaboration and helps produce higher-quality software with fewer defects.	
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation. Answer:- Incomplete test coverage occurs when some requirements, input conditions, or scenarios are not tested. Testers should identify coverage gaps by mapping requirements to test cases using a requirements traceability matrix. Test cases should include normal, invalid, boundary, and exceptional conditions. For example, a banking transaction should be tested for successful transactions, insufficient balance, invalid account details, cancelled transactions, and duplicate transactions. Coverage measurements can also be used to identify untested areas. Proper test coverage improves reliability and reduces the possibility of defects remaining undetected in the production system.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards. Answer:- Clear and structured defect reporting is important for quick identification and resolution of software problems. A defect report should include the defect ID, title, affected module, description, steps to reproduce, expected result, actual result, severity, and priority.	3



	Screenshots, logs, or other evidence can also be attached when required. For example, instead of simply writing “transaction failed,” the tester should provide the exact steps and input values that caused the failure. Clear reports help developers reproduce and fix defects quickly. Standardized defect reporting improves communication, tracking, and testing efficiency.	
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency. Answer:- Process-based testing ensures that a standard testing process is followed across all software modules. The organization should define common procedures for requirement analysis, test planning, test case design, execution, defect reporting, and regression testing. Standard test case templates, checklists, and defect-reporting formats should be used by all testers. Regular reviews can ensure that the defined testing process is followed correctly. For example, every module can use the same procedure for creating and executing test cases. Standardization reduces inconsistencies between testers and modules and improves overall software quality.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design. Answer:- Boundary testing is used to identify defects at the limits of input values. Edge cases are often missed when testers focus only on normal inputs, so values around the minimum and maximum limits should be tested. For example, if an application accepts an age from 18 to 60, the tester should test 17, 18, 19, 59, 60, and 61. This technique is called Boundary Value Analysis and is useful because many programming errors occur at boundaries. Test cases should include minimum, maximum, just-below, and just-above values. Including boundary conditions improves coverage and reduces failures after deployment.	3

19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices. Answer:- Proper defect documentation is necessary for tracking and managing software problems effectively. Each defect should contain information such as defect ID, module, description, severity, priority, steps to reproduce, expected result, actual result, cause, and resolution. A structured repository makes defects easier to search, track, and analyze throughout the software life cycle. Historical defect information can be used to identify recurring problems and high-risk modules. For example, repeated login defects can be analyzed and converted into additional regression test cases. Good documentation improves defect management and helps prevent similar defects in future releases.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves Answer:- Repeated transaction failures indicate that the banking system requires a more systematic testing approach. Testers should first analyze the requirements and identify important transaction scenarios. Test cases should cover successful transactions, failed transactions, cancelled transactions, duplicate transactions, invalid account details, insufficient balance, transaction limits, and network failures. Functional, integration, security, performance, and regression testing should be performed. All detected defects should be properly reported, fixed, and retested before release. A structured testing process improves defect detection, transaction reliability, system quality, and customer confidence.	3



quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



Code of Conduct:

I _____(P. Veeraswami / 192311253) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Veera swami

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand